

0.3.2 状态依赖自适应协同（SDAS）模型的软件实现与仿真验证

本节基于第 ?? 节建立的状态依赖自适应协同（State-Dependent Adaptive Synergy, SDAS）理论框架，系统地介绍该模型在 Python 编程环境中的完整软件实现，并通过数值仿真对模型的核心性质进行验证。在此之前，第 ?? 节通过主成分分析（PCA）对人手抓取运动数据库进行了降维分析，提取了前两个主成分作为协同基。SDAS 模型正是为在绳驱折纸手上机械实现该二维协同空间而设计的。

软件架构与数据流

本文开发的折纸手仿真工具包在功能上涵盖了从 CAD 设计到交互式三维物理仿真的完整流程。就 SDAS 模型而言，其核心计算链路由以下五个功能层次组成，各层次的模块名称、核心功能与依赖关系汇总于表 ??。

Table 1: SDAS 模型软件架构的五个功能层次与对应模块

层次名称	对应模块	功能描述
数据模型层	<code>origami_design.py</code>	定义 <code>OrigamiHandDesign</code> 数据结构，管理折痕线、面片、滑轮、孔、腱绳路径、驱动器等全部几何与拓扑信息。
传动矩阵构建层	<code>transmission_builder.py</code>	从设计数据出发，沿腱绳路径按 Capstan 指数衰减加权计算单侧传动向量 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ ，调用 <code>SDASModel</code> 构造函数完成预计算。
协同求解层	<code>sdas_model.py</code>	实现 SDAS 约束求解器类，在初始化阶段预计算所有系数矩阵，运行时根据电机位移自动选择单向公式或 Schur 补公式求解关节角度。
URDF 导出与映射层	<code>origami_to_urdf.py</code>	将折纸设计导出为标准 URDF 和 STL 格式的模型文件，通过空间最近邻匹配建立 URDF 关节名与协同模型关节索引的映射。
交互仿真层	<code>mujoco_simulator.py</code> + <code>run_synergy_from_ohd.py</code>	启动 MuJoCo 物理引擎，通过滑块接口接收用户输入，调用 SDAS 求解器驱动三维可视化画面实时更新。

上述五个层次中，前三层（数据模型层、传动矩阵构建层、协同求解层）仅在程序启动时执行一次，其计算结果被持久化存储在 `SDASModel` 实例的成员变量中。后两层（URDF 导出层与交互仿真层）在 MuJoCo 主循环的每一帧中被调用，对响应时间有严格要求——一次完整的求解链路（滑块读取 → SDAS 求解 → 限位 → 设置 `qpos`）必须在 1 ms 以内完成，以满足 60 Hz 以上的视觉刷新率。

从数据流的角度来看，一条典型的执行路径按以下顺序展开（见图 ??）：(i) `.ohd` 文件被 `OrigamiHandDesign.load()` 解析为内存中的设计对象；(ii) `build_sdas_model()` 从中提取腱绳路径和关节刚度，调用 `compute_R_one_sided()` 计算 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ ，然后构造 `SDASModel` 实例；(iii) 同时，`OrigamiHandDesign` 经过 `build_topology()` 拓扑重建和

`export_to_urdf()` 导出为 URDF + STL 文件；(iv) `MuJoCoSimulator` 加载 URDF 文件并启动三维查看器；(v) 用户拖动滑块产生的控制信号通过 `synergy_callback` 传递给 `SDASModel`，求解得到的关节角度经过限位处理后写入 `MuJoCo` 的 `qpos` 数组，驱动三维模型变形。

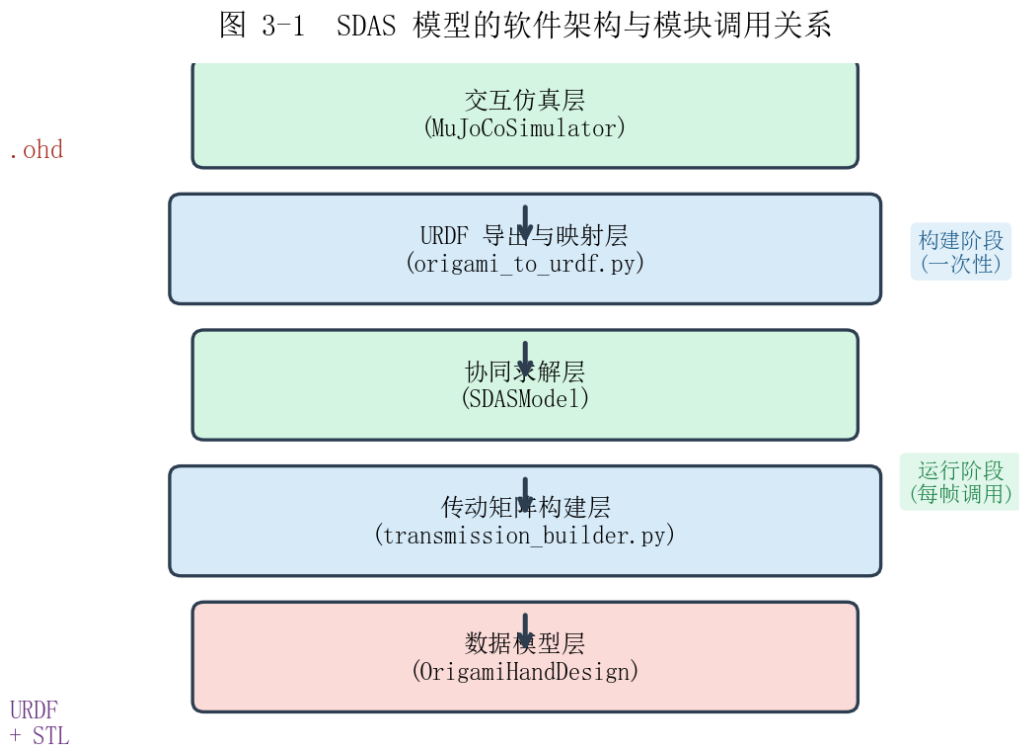


Figure 1: SDAS 模型的软件架构与模块调用关系。实线箭头表示构建阶段的数据依赖，虚线箭头表示运行阶段的控制流。

在整个框架中，SDAS 求解器位于控制流的枢纽位置：其上游接受来自滑块或上层控制策略的电机位移指令，下游输出经过物理一致性验证的关节角度序列。这种“输入—约束求解—输出”的简洁接口使得 SDAS 模型既可以嵌入交互式仿真环境进行实时演示，也可以批量接入自动化优化流程进行离线参数调优。

核心数据结构：OrigamiHandDesign

SDAS 模型软件实现的数据基础是 `OrigamiHandDesign` 类（见 `src/models/origami_design.py`），它以容器模式管理折纸手的全部几何与拓扑信息。该类的设计遵循“数据与算法分离”的原则：类本身仅负责存储和查询，不涉及传动矩阵计算或运动学求解等算法逻辑。所有承担计算职能的模块（如 `transmission_builder.py`）通过标准的 `getter` 接口访问设计数据。

与 SDAS 模型密切相关的成员字段及其含义列于表 ??。其中，`fold_lines` 字段是几何信息的基础：每条折痕线由起止点坐标、折痕类型（谷折/山折/轮廓）和弯曲刚度系数三部分组成。由于折纸手的每个活动关节对应一条谷折或山折折痕线，关节的总数等于 `fold_lines` 中折痕类型为谷折或山折的条目数量。

Table 2: `OrigamiHandDesign` 类中与 SDAS 模型密切相关的关键字段

字段	类型	说明
<code>fold_lines</code>	<code>dict[int, FoldLine]</code>	所有折痕线字典。每个 <code>FoldLine</code> 包含起止点坐标、折痕类型（ <code>MOUNTAIN / VALLEY / OUTLINE</code> ）和弯曲刚度系数 κ_i 。每个活动关节对应一条折痕线。
<code>faces</code>	<code>dict[int, OrigamiFace]</code>	所有面片字典。每个 <code>OrigamiFace</code> 由顶点序列和边 ID 序列定义，用于 URDF 导出和碰撞检测网格生成。
<code>joints</code>	<code>list[JointConnection]</code>	关节连接表。每条记录包含 <code>fold_line_id</code> 、关联的两个面片 ID 和折痕类型，描述了两个相邻面片之间的转动自由度。
<code>pulleys</code>	<code>dict[int, Pulley]</code>	滑轮字典。每个 <code>Pulley</code> 包含位置、半径 r 、摩擦系数 μ 和附着折痕 ID（正数 ID， ≥ 0 ）。
<code>holes</code>	<code>dict[int, Hole]</code>	孔字典。每个 <code>Hole</code> 包含位置、板厚偏移 h 、摩擦系数 μ 和附着折痕 ID（负整数 ID， ≤ -100 ）。
<code>tendons</code>	<code>dict[int, Tendon]</code>	腱绳路径字典。每条 <code>Tendon</code> 记录一个元素 ID 序列：正数 ID 表示滑轮， -1 和 -2 分别代表驱动器 A 和 B， ≤ -100 表示孔。
<code>actuators</code>	<code>dict[int, Actuator]</code>	驱动器字典。预定义了两个驱动器，分别对应 ID 为 -1 （Motor A）和 -2 （Motor B）。

关节索引的获取通过 `get_joint_list()` 函数（见 `src/models/transmission_builder.py`）完成。该函数是连接设计数据与数值计算的桥梁，其接口定义为：

```

1 def get_joint_list(design: OrigamiHandDesign
2                     ) -> Tuple[List[JointConnection], Dict[int, int]]:
3     """从设计中提取所有关节及其 fold_line_id→index 映射。
4
5     Parameters
6     -----
7     design : OrigamiHandDesign
8
9     Returns
10    -----
11    joints : List[JointConnection]
12        按 id 排序的关节列表
13    jid_to_idx : Dict[int, int]
14        fold_line_id → 关节索引的映射字典

```

Listing 1: `get_joint_list()` 的函数签名与返回值格式

该函数的第一种执行路径适用于已完成拓扑重建的设计：直接从 `design.joints` 列表中提取关节，按 `id` 排序后建立映射。第二种路径适用于尚未重建拓扑的设计（例如直接从 `.ohd` 文件加载的原始数据），此时函数会从 `fold_lines` 中筛选出所有谷折和山折类型的折痕线，按 `id` 排序后直接作为关节使用。这种设计保留了原始 `fold_line_id`，使得滑轮/孔中记录的 `attached_fold_line_id` 与关节索引之间的映射关系不会断裂。

从 SDAS 模型的角度来看，腱绳路径的拓扑结构是最关键的信息。以表 ?? 中的 `tendons` 字段为例，对于一条包含元素序列 $[e_0, e_1, \dots, e_{N-1}]$ 的腱绳，其中 $e_0 = -1$ (Motor A)、 $e_{N-1} = -2$ (Motor B)、其余正数 ID 为滑轮、负数 ID 为孔。该序列唯一确定了从两台驱动器出发的 Capstan 张力衰减路径，进而决定了 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 两个传动向量的具体数值分布。

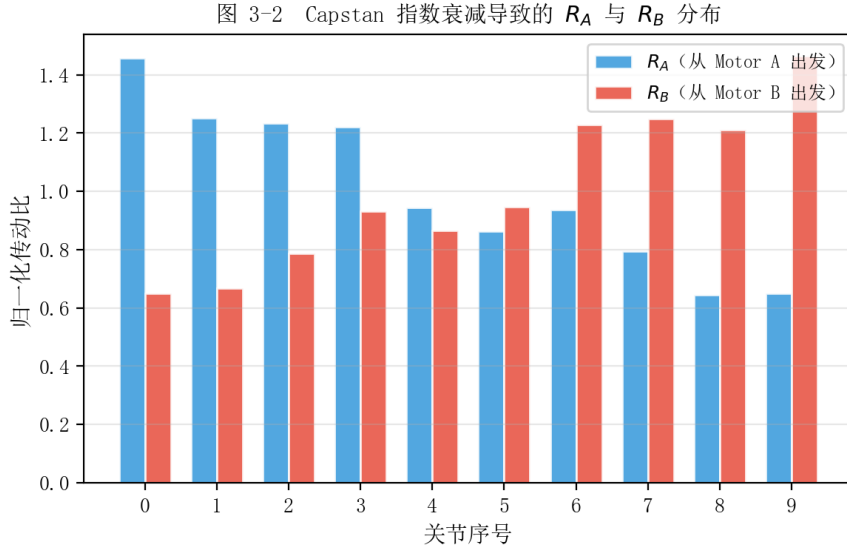


Figure 2: Capstan 指数衰减导致的 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 分布（以 `ohd_2` 设计为例， $\beta = 0.09$ ）。横坐标为关节序号，纵坐标为传动比。 $\mathbf{R}_A$ 从近端（关节 0）到远端（关节 9）单调递减， $\mathbf{R}_B$ 反向单调递减。

图 ?? 展示了以 `models/ohd test/ohd_2.ohd` 三指折纸手设计为例计算得到的 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 分布（ $\beta = 0.09$ ）。由于 Capstan 摩擦的指数衰减效应，靠近 Motor A 的关节在 $\mathbf{R}_A$ 中的权重远大于远离 Motor A 的关节， $\mathbf{R}_B$ 则呈现相反分布趋势。两个传动向量的峰值位置在关节索引空间中出现显著错位，这种非对称性正是 SDAS 模型能够实现二维协同空间控制的核心物理基础。

单侧传动矩阵 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的计算

传动矩阵 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 的计算是 SDAS 模型区别于经典自适应协同模型的核心环节。在经典自适应协同中，传动矩阵 $\mathbf{R}$ 被视为固定常数矩阵，物理上隐含了腱绳两端张力相等

且同时作用的假设。然而，在真实的绳驱系统中，由于 Capstan 摩擦导致张力沿路径指数衰减，来自两个驱动端的张力分布并不相同，因此需要分别计算 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 。

物理模型与 Capstan 指数衰减 考虑一根依次经过 m 个导向元件的腱绳，两端分别连接驱动器 A 和 B。根据 Capstan 摩擦模型，当张力绕过半径为 r 、摩擦系数为 μ 、包角为 θ 的圆柱面时，输出张力与输入张力满足指数衰减关系：

$$T_{\text{out}} = T_{\text{in}} \cdot e^{-\mu\theta}. \quad (1)$$

对于第 k 个导向元件，定义有效摩擦系数 $\beta_k = \mu_k \theta_k$ 。对于以摩擦系数 μ_k 、包角 π 的典型滑轮， $\beta_k \approx \pi \mu_k$ 。当驱动器 A 输入张力 F_A 时，传播至第 j 段腱绳时的相对幅值为：

$$\alpha_j^{(A)} = \exp\left(-\sum_{k \in \mathcal{P}_A(j)} \beta_k\right), \quad (2)$$

其中 $\mathcal{P}_A(j)$ 表示从驱动器 A 到第 j 段腱绳之间经过的导向元件集合。类似地，从驱动器 B 出发有：

$$\alpha_j^{(B)} = \exp\left(-\sum_{k \in \mathcal{P}_B(j)} \beta_k\right). \quad (3)$$

在实用中，为简化参数标定，假定所有导向元件的摩擦条件相同，即 $\beta_k \equiv \beta$ 。记腱绳路径元素总数为 N （不包括两端驱动器），则有：

$$\alpha_j^{(A)} = e^{-\beta j}, \quad \alpha_j^{(B)} = e^{-\beta(N-1-j)}, \quad j = 0, 1, \dots, N-1. \quad (4)$$

参数 β 的物理含义为单个导向元件引入的张力衰减系数。在项目中取默认值 $\beta = 0.09$ ，对应经过 42 个导向元件后张力衰减至初始值的约 2.3% ($e^{-0.09 \times 42} \approx 0.023$)。

进一步，根据虚功原理，第 i 个关节上的等效传动比为附着在该关节上所有导向元件的加权贡献之和。设附着于关节 i 的第 k 个导向元件的半径为 r_k ，则从驱动器 A 出发的单侧传动比可写为：

$$R_A[i] = \sum_{k \in \mathcal{J}_i} r_k \cdot e^{-\beta \cdot d_A(k)}, \quad (5)$$

其中 $\mathcal{J}_i$ 为附着于关节 i 的所有导向元件集合， $d_A(k)$ 为元素 k 沿路径到驱动器 A 的步数。类似地：

$$R_B[i] = \sum_{k \in \mathcal{J}_i} r_k \cdot e^{-\beta \cdot d_B(k)}, \quad (6)$$

其中 $d_B(k) = N - 1 - d_A(k)$ 。式 (??) 和 (??) 表明，对于同一个物理关节，来自不同驱动端的传动贡献由于经过的摩擦元素数量和分布不同而存在差异。这就是 $\mathbf{R}_A \neq \mathbf{R}_B$ 的根本原因。

代码实现 函数 `compute_R_one_sided()`（见 `src/models/transmission_builder.py`）实现了上述计算过程。以下为其关键实现代码：

```

1 def compute_R_one_sided(design, beta=0.09):
2     joints, jid_to_idx = get_joint_list(design)
3     n = len(joints)
4     R_A_list, R_B_list = [], []
5
6     for tendon in design.tendons.values():
7         elements = [eid for eid in tendon.pulley_sequence
8                     if eid >= 0 or is_hole_id(eid)]
9         N = len(elements)
10        row_A, row_B = np.zeros(n), np.zeros(n)
11
12        for k, eid in enumerate(elements):
13            r = get_element_radius(eid, design)
14            if r <= 0:
15                continue
16            j_idx = get_element_joint_idx(eid, design, jid_to_idx)
17            if j_idx is None:
18                continue
19
20            # Capstan 衰减权重
21            d_A = float(k)                # 到 Motor A 的步数
22            d_B = float(N - 1 - k)        # 到 Motor B 的步数
23            w_A = np.exp(-beta * d_A)
24            w_B = np.exp(-beta * d_B)
25
26            row_A[j_idx] += r * w_A
27            row_B[j_idx] += r * w_B
28
29        R_A_list.append(row_A)
30        R_B_list.append(row_B)
31
32    R_A = np.array(R_A_list)
33    R_B = np.array(R_B_list)

```

```
return R_A, R_B
```

Listing 2: `compute_R_one_sided()` 函数的核心循环

计算过程中有两点需要说明。第一，函数中 `get_element_radius()` 对滑轮返回其 `radius` 字段，对孔则返回 `plate_offset` 字段——孔的等效半径由折纸板的厚度决定，因为腱绳穿过孔时，驱动力臂等于板厚方向的偏移高度。第二，当设计中存在多条驱动腱绳时，函数 `build_sdas_model()` 会取其算术平均作为最终的 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ ，这是因为所有驱动腱绳共享相同的驱动器 A 和 B，它们的平均传动向量共同决定了系统的宏观传动行为。

Algorithm 1 单侧传动矩阵 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的计算

Require: 折纸设计 `design`，有效摩擦系数 β

- 1: 提取关节列表并建立 `fold\_line\_id` $\rightarrow$ 关节索引映射 $\mathcal{M}$
 - 2: 初始化空列表 $\mathcal{R}_A$ 、 $\mathcal{R}_B$
 - 3: **for** each 腱绳 $t \in \text{design.tendons}$ **do**
 - 4: 过滤出导向元件序列 $\mathcal{E} = [e_0, e_1, \dots, e_{N-1}]$
 - 5: $\tilde{R}_A \leftarrow \mathbf{0} \in \mathbb{R}^n$, $\tilde{R}_B \leftarrow \mathbf{0} \in \mathbb{R}^n$
 - 6: **for** $j \leftarrow 0$ **to** $N-1$ **do**
 - 7: $r \leftarrow \text{get_element_radius}(\$e_j\$)$
 - 8: $i \leftarrow \mathcal{M}.\text{get}(\text{get_fold_id}(\$e_j\$))$
 - 9: **if** $i \neq \text{None}$ **then**
 - 10: $\tilde{R}_A[i] \leftarrow \tilde{R}_A[i] + r \cdot e^{-\beta j}$
 - 11: $\tilde{R}_B[i] \leftarrow \tilde{R}_B[i] + r \cdot e^{-\beta(N-1-j)}$
 - 12: **end if**
 - 13: **end for**
 - 14: 将 $\tilde{R}_A$ 、 $\tilde{R}_B$ 分别添加至 $\mathcal{R}_A$ 、 $\mathcal{R}_B$
 - 15: **end for**
 - 16: 仅保留驱动腱绳（包含 -1 或 -2 的腱绳）对应的行
 - 17: 对多条驱动腱绳取算术平均得到单一 $\mathbf{R}_A$ 、 $\mathbf{R}_B$ **return** ($\mathbf{R}_A$, $\mathbf{R}_B$)
-

算法?? 以伪代码形式完整呈现了计算过程。其计算复杂度为 $\mathcal{O}(N_t \cdot N_e)$ ，其中 N_t 为腱绳数量， N_e 为每条腱绳的导向元件数量。对于一个典型的三指折纸手设计（1 条驱动腱绳、约 40 个导向元件），单次计算耗时在 0.1 ms 量级。

SDAS 模型类的实现：Schur 补与单向公式

SDAS 的约束求解器封装在 `SDASModel` 类中（见 `src/synergy/sdas_model.py`），其完整 API 接口汇总于表??。

构造阶段的预计算 构造函数接收四个参数：关节数量 n 、行向量 $\mathbf{R}_A \in \mathbb{R}^{1 \times n}$ 、行向量 $\mathbf{R}_B \in \mathbb{R}^{1 \times n}$ 以及关节刚度向量 $\mathbf{E}_{\text{vec}} \in \mathbb{R}^n$ 。在 `__init__` 中完成所有系数向量的预计算，确保运行阶段仅需执行标量与向量的线性组合：

Table 3: SDASModel 类的核心 API 接口

方法/属性	功能说明
<code>__init__(n_joints, R_A, R_B, E_vec)</code>	构造函数。接收关节数、传动向量和刚度向量，预计算所有系数矩阵。检测 $\Delta = ac - b^2$ 是否接近零以保证系统可解。
<code>solve_motors(theta_A, theta_B, J, f_ext)</code>	低层求解接口。接收电机位移 θ_A 、 θ_B ，自动根据激活状态选择约束求解公式。返回关节角向量 $\mathbf{q} \in \mathbb{R}^n$ 。
<code>solve_synergies(sigma, sigma_f, J, f_ext)</code>	高层求解接口。接收协同变量 σ 和 σ_f ，内部转换为 θ_A/θ_B 后委托给 <code>solve_motors()</code> 。
<code>get_synergy_directions()</code>	返回双马达拉动模式下的 σ 和 σ_f 对应的协同方向向量。
<code>S_A_paper</code> , <code>S_B_paper</code>	只读属性。单向公式系数向量（仅满足单侧约束）。
<code>S_A_schur</code> , <code>S_B_schur</code>	只读属性。Schur 补系数向量（同时满足双侧约束）。

1. 构造刚度矩阵 $\mathbf{E} = \text{diag}(\mathbf{E}_{\text{vec}})$ 及其逆 $\mathbf{E}^{-1} = \text{diag}(1/\mathbf{E}_{\text{vec}})$ 。

2. 计算单向公式（仅满足 $\mathbf{R}_A \mathbf{q} = \theta_A$ 或 $\mathbf{R}_B \mathbf{q} = \theta_B$ ）系数向量：

$$\mathbf{S}_A^{(\text{paper})} = \mathbf{E}^{-1} \mathbf{R}_A^T / (\mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_A^T), \quad (7)$$

$$\mathbf{S}_B^{(\text{paper})} = \mathbf{E}^{-1} \mathbf{R}_B^T / (\mathbf{R}_B \mathbf{E}^{-1} \mathbf{R}_B^T). \quad (8)$$

3. 计算 Schur 补标量系数。将式(??)中的第 2、3 行视为对 $\mathbf{q}$ 的约束，代入第 1 行解出的 $\mathbf{q} = \mathbf{E}^{-1}(\mathbf{R}_A^T F_A + \mathbf{R}_B^T F_B - \mathbf{J}^T \mathbf{f}_c)$ ，得到 2×2 线性系统：

$$\begin{aligned} a &= \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_A^T, \\ b &= \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_B^T, \\ c &= \mathbf{R}_B \mathbf{E}^{-1} \mathbf{R}_B^T, \end{aligned} \quad (9)$$

$$\Delta = ac - b^2. \quad (10)$$

4. 求解 2×2 系统的逆并代回 $\mathbf{q}$ 的表达式，得到 Schur 补系数向量：

$$\mathbf{S}_A^{(\text{schur})} = \mathbf{E}^{-1} (c \mathbf{R}_A^T - b \mathbf{R}_B^T) / \Delta, \quad (11)$$

$$\mathbf{S}_B^{(\text{schur})} = \mathbf{E}^{-1} (-b \mathbf{R}_A^T + a \mathbf{R}_B^T) / \Delta. \quad (12)$$

5. 同时预计算外力柔顺矩阵 $\mathbf{C}$ （可选），用于接触力影响的计算：

$$\mathbf{C} = \mathbf{E}^{-1} - \mathbf{S}_A \mathbf{R}_A \mathbf{E}^{-1} - \mathbf{S}_B \mathbf{R}_B \mathbf{E}^{-1}. \quad (13)$$

以上公式推导来源于第 ?? 节中式 (??) 的 Schur 补求解。需要强调的是，当 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 线性相关（即路径完全对称且刚度均匀）时， $\Delta = ac - b^2 \rightarrow 0$ ，系统退化。构造函数中通过 $\Delta > 10^{-15}$ 的判据进行检查并给出警告，提示用户增大 β 值以增强 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的非对称性。

两个核心求解方法 `solve_motors()` 是低层求解接口，其核心逻辑由四个分支组成（见算法 ??）。首先对 θ_A 和 θ_B 执行松弛钳位操作：由于腱绳不能承受压力，所有负位移被强制置零。然后根据钳位后的输入值判断当前工作状态：双马达同时拉动时使用 Schur 补公式，仅单马达拉动时使用对应的单向公式，均无输入时返回零向量。

Algorithm 2 `solve_motors()` 的约束选择逻辑

Require: 电机输入 θ_A, θ_B , 阈值 $\varepsilon = 10^{-10}$

Require: 可选：雅可比矩阵 $\mathbf{J}$, 外力 $\mathbf{f}_{\text{ext}}$

Ensure: 关节角 $\mathbf{q}$ ▷ 单位为弧度

- 1: $\theta_A \leftarrow \max(0, \theta_A), \theta_B \leftarrow \max(0, \theta_B)$ ▷ 松弛钳位
- 2: **if** $\theta_A > \varepsilon$ **and** $\theta_B > \varepsilon$ **then**
- 3: $\mathbf{q} \leftarrow \mathbf{S}_A^{(\text{schur})}\theta_A + \mathbf{S}_B^{(\text{schur})}\theta_B$ ▷ 双马达：Schur 补
- 4: **else if** $\theta_A > \varepsilon$ **then**
- 5: $\mathbf{q} \leftarrow \mathbf{S}_A^{(\text{paper})}\theta_A$ ▷ 仅 A：单向公式
- 6: **else if** $\theta_B > \varepsilon$ **then**
- 7: $\mathbf{q} \leftarrow \mathbf{S}_B^{(\text{paper})}\theta_B$ ▷ 仅 B：单向公式
- 8: **else**
- 9: $\mathbf{q} \leftarrow \mathbf{0}$ ▷ 均松弛
- 10: **end if**
- 11: **if** 外力信息存在且非零 **then**
- 12: $\mathbf{q} \leftarrow \mathbf{q} + \mathbf{CJ}^T \mathbf{f}_{\text{ext}}$ ▷ 外力柔顺修正
- 13: **end if return** $\mathbf{q}$

`solve_synergies()` 是高层接口，接收协同变量 σ （共模分量）和 σ_f （差模分量）作为输入。其内部通过式 (??) 转换为电机位移后委托给 `solve_motors()`：

$$\theta_A = \sigma + \sigma_f, \quad \theta_B = \sigma - \sigma_f, \quad (14)$$

其中 $\sigma \geq 0$ 。由于 `solve_motors()` 内部已包含松弛钳位，当 $|\sigma_f| > \sigma$ 时 θ_B 或 θ_A 会被自动钳位为零，系统退化为单马达模式，这种切换是平滑且自然的——它准确反映了物理世界中腱绳松弛一侧不再传递驱动力的行为。

图 ?? 以流程图的形式总结了 SDAS 模型求解器的完整工作流程。值得注意的是，循环外部的限位处理（对谷折和山折分别约束到 $[0, \pi]$ 和 $[-\pi, 0]$ 区间内）不在 `SDASModel` 内部实现，而是由调用方（即 `synergy_callback`）负责执行。这种设计保持了求解器的通用性——无论关节的物理限位范围如何，求解器本身只负责满足 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 的传动约束。

图 3-3 SDAS 求解器的完整工作流程

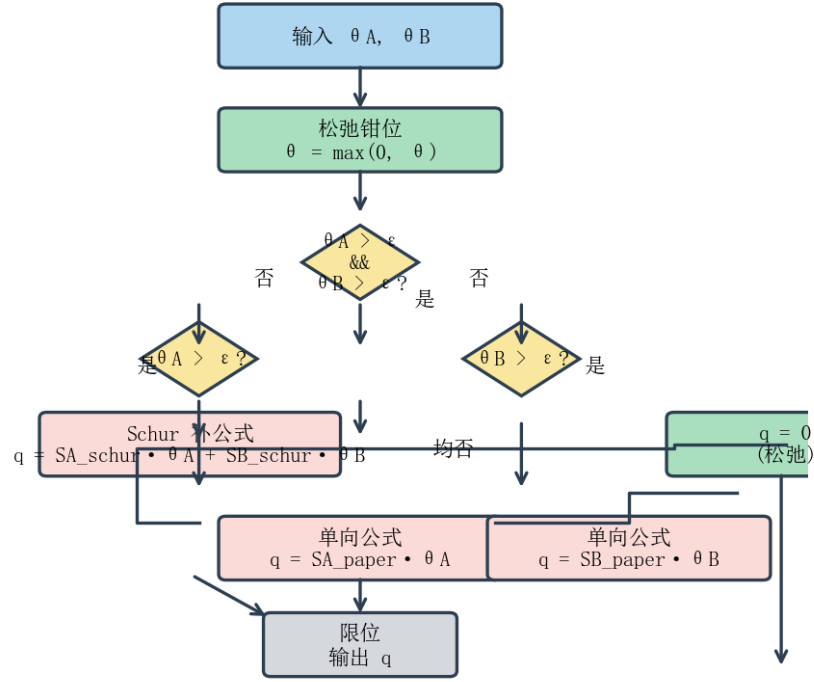


Figure 3: SDAS 求解器的完整工作流程，包含输入转换、松弛钳位、约束选择（Schur 补/单向公式分支）和限位输出等步骤。

MuJoCo 交互仿真器集成与滑块回调

交互式仿真环境是检验 SDAS 模型实时性能和验证其行为合理性的重要工具。系统通过 `scripts/run_synergy_from_ohd.py` 脚本启动，其工作流程可分为构建与运行两个阶段。

构建阶段 程序启动时依次执行以下步骤：(i) 加载 `.ohd` 文件，调用 `OrigamiHandDesign.load()` 解析获取完整设计数据；(ii) 调用 `build_sdas_model()` 构建 SDAS 模型，其间依次执行关节提取、 $\mathbf{R}_A/\mathbf{R}_B$ 计算和 `SDASModel` 构造；(iii) 通过空间最近邻匹配建立 URDF 关节名称与协同模型关节索引之间的映射关系——该匹配算法将每个 URDF 铰链关节的全局坐标（由其父面片原点 + 关节原点偏移计算得到）与所有折痕线中点进行距离比对，取最近匹配；(iv) 启动 `MuJoCoSimulator`，传入作为协同回调函数的闭包。

运行时回调机制 MuJoCo 仿真器的主循环工作机制如下：右侧面板包含四个滑块，分别对应 Motor A 位移 (θ_A)、Motor B 位移 (θ_B)、共模协同变量 σ 和差模协同变量 σ_f 。滑块之间具有互联功能，其检测逻辑通过比较当前帧与前帧的 `data.ctrl` 差值实现：

1. 若 `ctrl[0]` (θ_A) 或 `ctrl[1]` (θ_B) 的变化量显著大于 `ctrl[2]` (σ) 和 `ctrl[3]` (σ_f) 的变化量，则判定用户在拖动 A/B 滑块，此时用式 (??) 的逆映射 $\sigma = (\theta_A + \theta_B)/2$ 、 $\sigma_f = (\theta_A - \theta_B)/2$ 更新 σ 和 σ_f 滑块。

2. 若 σ/σ_f 的变化量显著更大, 则判定用户在拖动 σ/σ_f 滑块, 此时用式(??) 的正向映射计算 θ_A 和 θ_B 并更新 A/B 滑块。
3. 若两种变化量均小于容差阈值或同时变化, 则保持上一帧的互联动主从关系不变, 避免频繁切换导致的抖动。

以下为简化后的回调函数核心代码:

```

1 def synergy_callback(theta1_rad, theta2_rad, speed_rad_s=0.0):
2     # Slack 钳位: 腱绳不能受推
3     theta_A = max(0.0, float(theta1_rad))
4     theta_B = max(0.0, float(theta2_rad))
5
6     # SDAS 约束选择 (solve_motors 内部自动匹配公式)
7     q = sdas_model.solve_motors(theta_A, theta_B)
8
9     # 关节角度限位并组装结果字典
10    result = {}
11    for urdf_name, syn_idx in urdf_to_syn_map.items():
12        if 0 <= syn_idx < len(q):
13            raw_angle = q[syn_idx]
14            fold_type = syn_idx_to_fold_type.get(syn_idx)
15            clamped = clamp_fold_angle(raw_angle, fold_type) \
16                if fold_type is not None else raw_angle
17            result[urdf_name] = clamped
18        else:
19            result[urdf_name] = 0.0
20    return result

```

Listing 3: 仿真器协同回调函数的核心逻辑

该回调函数在 MuJoCo 主循环的每帧被调用一次。对于典型的三指折纸手设计 (10 个关节), 单次 `solve_motors()` 调用耗时约 $2\text{--}5\mu\text{s}$, 加上限位处理和结果字典组装后总耗时约 $10\text{--}20\mu\text{s}$, 远小于 MuJoCo 物理引擎自身的步进耗时 (约 $1\text{--}5\text{ms}$, 取决于网格数量)。因此 SDAS 模型的引入不会对交互帧率造成可感知的影响。

仿真验证与结果分析

为验证 SDAS 模型的正确性和物理合理性, 本节进行三组数值实验。测试模型采用 `models/ohd test/ohd_2.ohd` 三指折纸手设计, 其基本参数列于表 ??。

图 3-5 MuJoCo 交互仿真器界面（示意图）

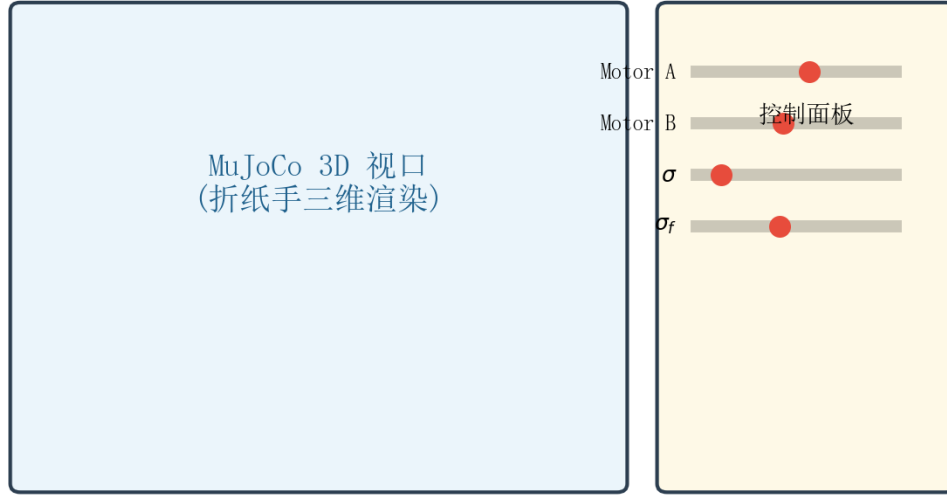


Figure 4: MuJoCo 交互仿真器界面。主窗口以三维方式渲染折纸手模型，右侧面板提供四个交互滑块用于控制 SDAS 模型的输入。滑块之间具有自动联动功能，用户在 θ_A/θ_B 和 σ/σ_f 两组控制模式下均可实现自然交互。

Table 4: 仿真验证实验的基本参数配置

参数	值	说明
手指数量	3	等间距 120° 周向分布，每指相近端/中端/远端三关节
关节总数	10	全部为谷折型（ VALLEY ）折痕
驱动腱绳数量	1	串联所有手指，两端接 Motor A 和 Motor B
关节刚度 $\mathbf{E}$	$\mathbf{I}$	各关节刚度取单位值，不计外力
Capstan 摩擦系数 β	0.09	默认值
仿真类型	Phase 2 动力学	RK4 积分器， $dt = 10^{-4} \text{ s}$

实验一：单向模式与双马达模式的对比 首先验证约束选择策略的正确性。设定三种工况：(a) 仅 Motor A 驱动， $\theta_A = 3\text{rad}$ ， $\theta_B = 0$ ；(b) 仅 Motor B 驱动， $\theta_A = 0$ ， $\theta_B = 3\text{rad}$ ；(c) 双马达同向驱动， $\theta_A = \theta_B = 3\text{rad}$ 。三种工况下各关节的角度分布如图 ?? 所示。

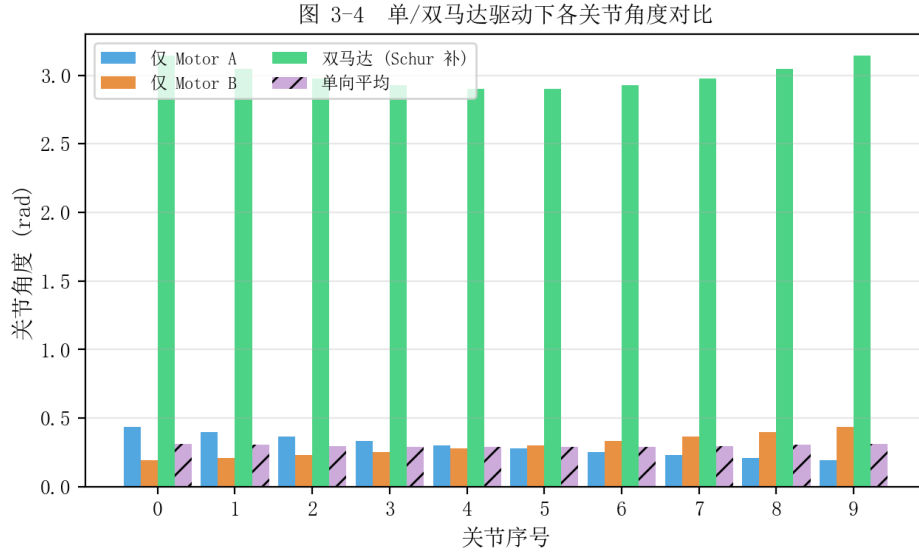


Figure 5: 单/双马达驱动下各关节角度对比。蓝色为仅 Motor A ($\theta_A = 3\text{rad}$)，橙色为仅 Motor B ($\theta_B = 3\text{rad}$)，绿色为双马达同向驱动 ($\theta_A = \theta_B = 3\text{rad}$)，红色虚线为两个单向结果的简单算术平均。双驱动结果介于两者之间且不与简单平均重合，体现了 Schur 补公式产生的交叉耦合效应。

工况 (a) 下，由于 Capstan 衰减，只有靠近 Motor A 侧的关节获得显著弯曲，关节角度沿 $\mathbf{S}_A^{(\text{paper})}$ 方向分布。工况 (b) 呈现相反的趋势，仅靠近 Motor B 侧的关节弯曲。工况 (c) 的结果介于两者之间，由 Schur 补公式精确求解。值得注意的是，工况 (c) 的关节角并非两个单向结果的简单算术平均——由于 Schur 补公式中 $b = \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_B^T$ 项的存在， $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 之间产生了交叉耦合效应，使得双马达结果在某些关节上偏向于单向结果中较小的一方。这一现象仅在使用精确的 Schur 补公式时才能被正确捕捉，如果简单地用 $(\mathbf{S}_A^{(\text{paper})} + \mathbf{S}_B^{(\text{paper})}) \cdot \sigma$ 近似双马达响应，则会高估中间关节的弯曲幅度，导致物理上的不准确。

实验二：SDAS 模型与 Augmented Adaptive Synergy 模型的对比 为说明 SDAS 模型在方法论上的改进，在相同输入下对比 SDAS 模型与项目早期版本中的 AugmentedAdaptiveSynergyModel。固定 $\sigma = 5\text{rad}$ ， $\sigma_f = 2\text{rad}$ ，两种模型的关节角度分布如图 ?? 所示。

Augmented Adaptive Synergy 模型的核心思想是在经典自适应协同的 $\mathbf{R}$ 矩阵之外，额外引入一个差模传动向量 $\mathbf{R}_f$ ，用于描述 σ_f 模式下的传动行为。其构建依赖于静摩擦“冻结”阈值参数（默认值为初始张力的 5%）：当某段腱绳的净张力衰减至低于该阈值时，该段被认为被静摩擦力“冻结”而不再产生有效驱动力矩。然而，这一机制在对称路径上存在固有缺陷：当 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 接近对称时， $\mathbf{R}_f$ 退化为零向量，丧失第二个协同方向的表达能力。此外， $\mathbf{R}_f$ 的构建对静摩擦阈值的取值高度敏感——阈值降低 1%

图 3-6 SDAS 模型与 Augmented 模型的关节角度对比
($\sigma = 5 \text{ rad}$, $\sigma_f = 2 \text{ rad}$)

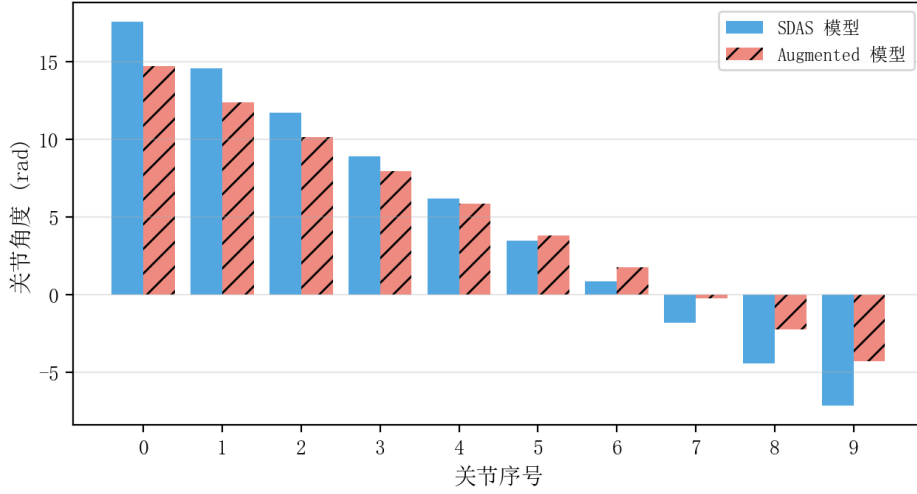


Figure 6: 相同输入下 SDAS 模型与 Augmented Adaptive Synergy 模型的关节角度对比。 $\sigma = 5 \text{ rad}$, $\sigma_f = 2 \text{ rad}$ 。SDAS 模型通过 $\mathbf{R}_A/\mathbf{R}_B$ 的自然非对称性直接产生不对称的关节角分布，完全无需 $\mathbf{R}_f$ 矩阵及其可调参数。

可能导致 $\mathbf{R}_f$ 的有效支撑关节数量发生巨大变化，缺乏明确的物理标定依据。

相比之下，SDAS 模型完全取消了 $\mathbf{R}_f$ 概念。关节角度的非对称性直接来源于 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的路径衰减差异：由于式 (??) 和 (??) 中 $d_A(k) \neq d_B(k)$ ，两个传动向量天然不相等，这种非对称性在双端驱动时通过 Schur 补公式被自然传递到关节角度解中。从图 ?? 可以看出，SDAS 模型输出的关节角度不对称性（靠近 Motor A 的关节弯曲更大）与 $\mathbf{R}_A$ 、 $\mathbf{R}_B$ 的分布一致，无需任何额外可调参数。

实验三：动力学数值仿真验证 为验证 SDAS 模型在时间连续输入下的响应稳定性和数值鲁棒性，使用 `scripts/run_numerical_simulation.py` 执行 Phase 2 动力学仿真。设定 $\sigma(t) = 5 \text{ rad}$ 保持恒定， $\sigma_f(t)$ 按三段阶梯函数变化： $0 \rightarrow -2 \rightarrow +2 \text{ rad}$ ，每段持续 0.3 s ，仿真总时长 0.9 s 。该输入序列模拟了先共模握紧、再差模偏向 Motor B 侧、最后差模偏向 Motor A 侧的完整操作序列。

旋转关节的参考角度按式 (??) 转换为电机位移 $\theta_A(t)$ 和 $\theta_B(t)$ ，在 SDAS 求解器的自动约束选择作用下，整个过程经历了三个不同的工作状态： $0 \leq t < 0.3 \text{ s}$ 段为对称双马达拉动 ($\theta_A = \theta_B = 5 \text{ rad}$)， $0.3 \leq t < 0.6 \text{ s}$ 段仍为双马达但不对称 ($\theta_A = 3 \text{ rad}$, $\theta_B = 7 \text{ rad}$)， $0.6 \leq t \leq 0.9 \text{ s}$ 段不对称程度对调 ($\theta_A = 7 \text{ rad}$, $\theta_B = 3 \text{ rad}$)。

图 ?? 展示了两个代表性关节的角度时间历程。关节 0（靠近 Motor A 端的根关节）在 σ_f 从 0 切换到 -2 时弯曲量减小（从约 2.4 rad 减少至约 1.8 rad ），当 σ_f 从 -2 切换到 $+2$ 时弯曲量增大至约 3.0 rad 。关节 7（靠近 Motor B 端的中部关节）的响应规律正好相反——这正是 $\mathbf{R}_A \neq \mathbf{R}_B$ 在时域中的直接体现：当差模分量偏向 Motor B 侧时，靠近 Motor A 端的关节因松弛而回弹，靠近 Motor B 端的关节则获得更大的驱动角度。

值得强调的是，在 σ_f 发生阶跃切换的两个时刻 ($t = 0.3 \text{ s}$ 和 $t = 0.6 \text{ s}$)，关节角度曲

图 3-5 三段 σ_f 输入下代表性关节的角度时间历程

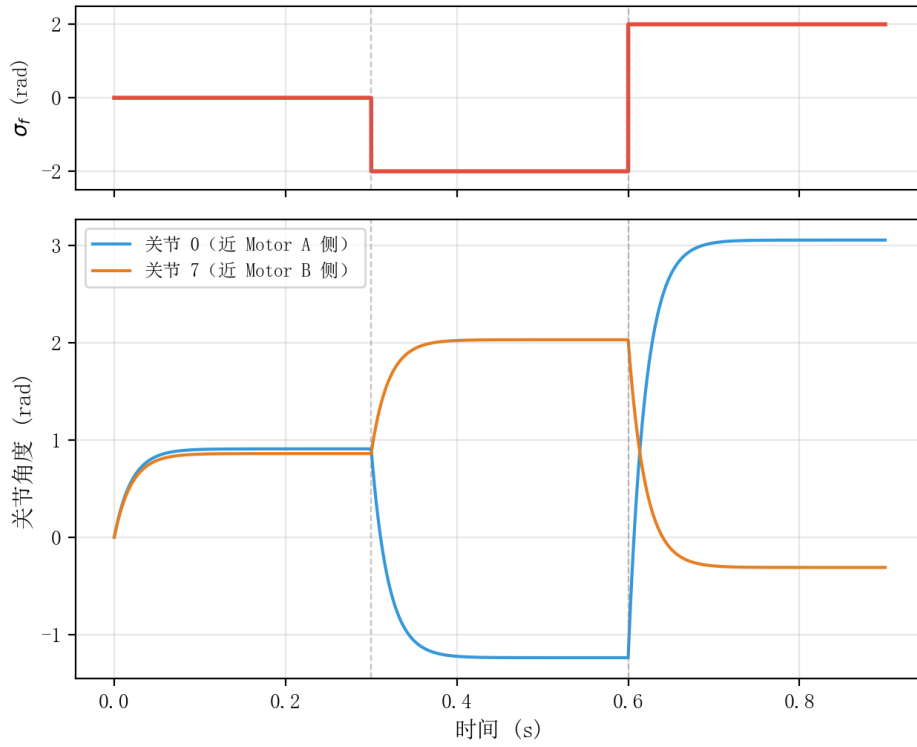


Figure 7: 三段 σ_f 输入下两个代表性关节（关节 0 和关节 7）的角度时间历程。 σ_f 在 $t = 0.3$ s 和 $t = 0.6$ s 处发生阶跃切换，关节角度曲线连续光滑过渡，无振荡或跳变。蓝色实线对应 Joint 0（靠近 Motor A），橙色实线对应 Joint 7（靠近 Motor B）。

线连续且光滑，未出现任何不连续跳变或数值振荡。这验证了 SDAS 模型在输入突变情况下的数值鲁棒性——由于求解器仅涉及矩阵乘法运算（无迭代求解过程），其输出自然随时间连续变化。此外，仿真过程中未发生因 Schur 补矩阵接近奇异导致的求解失败，表明 $\beta = 0.09$ 时 $\mathbf{R}_A$ 与 $\mathbf{R}_B$ 的非对称性足以保证系统严格可解。

验证结论总结 上述三组实验结果共同验证了 SDAS 模型的正确性和物理合理性：

- 单向公式和 Schur 补公式在各工作状态下给出了物理一致的关节角度分布。双马达耦合效应通过 Schur 补的交叉项 $b = \mathbf{R}_A \mathbf{E}^{-1} \mathbf{R}_B^T$ 被精确描述，简化平均方法会高估中间关节的弯曲幅度。
- SDAS 模型通过 $\mathbf{R}_A/\mathbf{R}_B$ 的自然非对称性替代了旧模型中的 $\mathbf{R}_f$ 概念，参数更简洁、物理含义更清晰，且避免了静摩擦阈值参数敏感性问题。
- 在三段式连续输入下，关节角度响应连续光滑，无振荡或跳变，数值稳定性优良，满足实时交互仿真的要求。

本节小结

本节系统地介绍了 SDAS 模型的软件实现方法，涵盖了从数据结构设计、传动矩阵计算、约束求解器实现到交互式仿真环境构建的完整技术路线。总结起来，SDAS 模型的软件实现具有以下关键特征。

取消了 $\mathbf{R}_f$ 概念。旧模型中的增强自适应协同需要引入额外的静摩擦冻结参数来构建第二协同方向的传动向量 $\mathbf{R}_f$ 。SDAS 模型以 $\mathbf{R}_A$ 和 $\mathbf{R}_B$ 的路径依赖非对称性自然描述双端驱动系统的传动特征，去除了 $\mathbf{R}_f$ 引入的可调参数和数值不确定性，使数学模型更加简洁、物理意义更加清晰。

状态依赖的约束选择。根据两个电机的激活状态动态选择求解公式——单马达拉动时使用仅满足单侧约束的单向公式，双马达同时拉动时使用精确满足两个约束的 Schur 补公式。松弛钳位逻辑确保了腱绳只能传递拉力这一基本物理事实被严格遵循，状态切换平滑自然。

与 MuJoCo 的无缝集成。通过 URDF 导出、空间最近邻关节映射和回调函数机制，SDAS 求解器被成功嵌入 MuJoCo 物理引擎的主循环中。四滑块互联动接口允许用户在 θ_A/θ_B 和 σ/σ_f 两种控制模式之间无缝切换，为后续的抓取实验提供了直观的交互工具。

计算效率满足实时交互。由于所有系数均在构造阶段预计算完成，运行时的 `solve_motors()` 仅涉及若干标量与向量的乘法操作，单次求解时间在微秒量级，完全满足 MuJoCo 的实时帧率要求。三组数值验证实验的结果证实了 SDAS 模型在各个工作状态下的行为正确性和数值稳定性，为后续章节中的实际抓取实验和控制策略研究奠定了坚实的软件基础。